



A fast algorithm for the all-pairs suffix–prefix problem[☆]

Jihyuk Lim, Kunsoo Park^{*}

Department of Computer Science and Engineering, Seoul National University, Seoul 08826, Republic of Korea

ARTICLE INFO

Article history:

Received 23 February 2017

Received in revised form 7 July 2017

Accepted 16 July 2017

Available online 25 July 2017

Keywords:

All-pairs suffix–prefix problem

Algorithm engineering

DNA sequence assembly

ABSTRACT

The all-pairs suffix–prefix problem occurs as a subproblem of DNA sequence assembly where it is the most time-consuming part of the whole assembly. Although there are algorithms for the all-pairs suffix–prefix problem which are optimal in the asymptotic time complexity, they are slower than SOF and Readjoinder which are state-of-the-art algorithms used in practice. In this paper we present an algorithm for the all-pairs suffix–prefix problem that uses a simple data structure for storing input strings and advanced algorithmic techniques for matching, which together lead to fast running time in practice. Our algorithm is 14 times faster than SOF and 18 times faster than Readjoinder on average in real datasets and random datasets.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

The all-pairs suffix–prefix (APSP) problem is defined as follows: Given a collection S of k strings $S_1, S_2, \dots, S_k$, find the longest suffix of S_i which is a prefix of S_j for all pairs S_i and S_j . Let N be the sum of lengths of the input strings $S_1, S_2, \dots, S_k$. The all-pairs suffix–prefix problem occurs as a subproblem of DNA sequence assembly where it is the most time-consuming part of the whole assembly. In DNA sequence assembly, a parameter om (for overlap minimum) is given for the APSP problem where we want to find the longest overlap of S_i and S_j whose length is at least om . The output of the APSP problem can be stored in a $k \times k$ matrix OV , where $OV[i, j]$ is the length of the longest suffix of S_i that is a prefix of S_j . Alternatively, the output can be a list of three integers $(i, j, OV[i, j])$ such that $OV[i, j] \geq om$ as a compact representation.

The all-pairs suffix–prefix problem has been studied in the fields of stringology and bioinformatics. In general, a solution for the APSP problem consists of two phases: the first phase is to build a data structure which represents all prefixes of the input strings, and the second matching phase is to search the data structure to find occurrences of suffixes of each input string (or equivalently, one can build a data structure representing all suffixes of the input strings, and search it for prefixes of each string). Gusfield et al. [1] proposed a novel algorithm of optimal $O(N + k^2)$ time for the APSP problem by building a generalized suffix tree for the input strings. Ohlebusch and Gog [2] gave another $O(N + k^2)$ -time algorithm by building an enhanced suffix array [3,4] for the input strings, which improves upon Gusfield et al.'s in running time and space. Tustumi et al. [5] further improved the running time and space of Ohlebusch and Gog's algorithm. Louza et al. [6] presented a parallel algorithm for the APSP problem which is based on Tustumi et al.'s.

[☆] A preliminary version of this paper was presented at the 16th International Symposium on Experimental Algorithms (SEA 2017). This research was supported by Collaborative Genome Program for Fostering New Post-Genome industry through the National Research Foundation of Korea (NRF) funded by the Ministry of Science ICT and Future Planning (No. NRF-2014M3C9A3063541).

^{*} Corresponding author.

E-mail addresses: jhlim@theory.snu.ac.kr (J. Lim), kpark@theory.snu.ac.kr (K. Park).

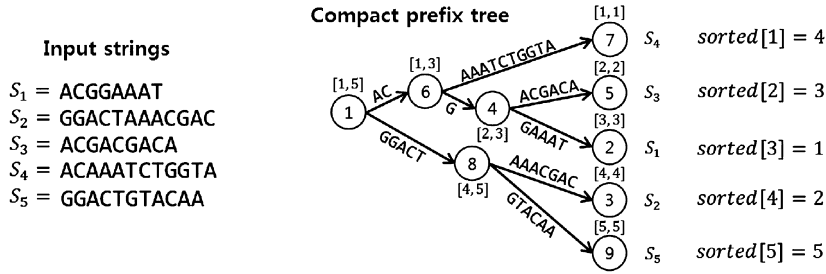


Fig. 1. Compact prefix tree of five strings.

In bioinformatics too, many algorithms have been proposed for the APSP problem [7–15], and Readjoiner [8] and SOF [13] are state-of-the-art algorithms which show best performances in practice. Although the algorithms in [1,2,5] are optimal in the asymptotic worst-case time complexity, Readjoiner and SOF are faster than these algorithms in practice [5,6]. Hence there is a mismatch between theoretical results and practice. A main reason for the mismatch is that the generalized suffix tree [1,16] and even the enhanced suffix array [3,4] are heavy machineries (though they provide powerful functionalities) and so the constants hidden in the asymptotic notations are quite big. On the other hand, SOF uses a simple but effective data structure called the compact prefix tree (also known as compact trie) for the input strings and its matching phase uses quite naive algorithmic techniques.

In this paper we propose a fast algorithm for the APSP problem. We first build a compact prefix tree for the input strings, but in the matching phase we need more advanced techniques because the compact prefix tree has limited functionalities, compared to the generalized suffix tree and the enhanced suffix array. We divide the matching phase into three cases depending on the lengths of suffixes of an input string, and in each case we use an appropriate algorithmic technique which finds efficiently occurrences of the suffixes of an input string corresponding to the case. We did experiments to compare our algorithm against SOF and Readjoiner with real datasets and random datasets. In the experiments our algorithm is 14 times faster than SOF and 18 times faster than Readjoiner on average. We also obtain reasonable scalability with a parallel implementation of our algorithm.

As another approach to the APSP problem, we can build the Aho–Corasick automaton [17] for the input strings, and solve the APSP problem by searching the automaton for each input string. We implemented this approach, but the Aho–Corasick automaton is another heavy machinery and just building it (without the matching phase) takes more time than the whole SOF. This comparison will be shown in Section 3.

This paper is organized as follows. In Section 2, we describe our algorithm in details. We show experimental results in Section 3, and conclude in Section 4.

2. New algorithm for the APSP problem

Let S be a string of length n over an alphabet Σ . We denote the length of S by $|S|$. The i -th character of S is denoted by $S[i]$ ($1 \leq i \leq |S|$), and a substring $S[i]S[i+1] \dots S[j]$ by $S[i..j]$. A substring $S[1..i]$ for $1 \leq i \leq n$ is called a prefix of S and a substring $S[i..n]$ for $1 \leq i \leq n$ is called a suffix of S . For strings A and B , we use $A < B$ to denote that A is lexicographically smaller than B .

We describe the compact prefix tree CT of k input strings defined in [13]. The compact prefix tree is basically a compact trie, i.e., there is one leaf corresponding to each input string and every internal node has at least two children. An array $sorted$ stores the lexicographic ordering of the input strings such that $S_{sorted[1]} \leq S_{sorted[2]} \leq \dots \leq S_{sorted[k]}$. The leaves of CT are in the lexicographic order, and each node v of CT has an interval $[a, b]$ such that $S_{sorted[a]}, \dots, S_{sorted[b]}$ are the leaves in the subtree rooted at v . The compact prefix tree provides a function $Find(s)$, which returns the node v nearest to the root such that s is a prefix of the string on the path from the root to v . If such a node does not exist, $Find(s)$ returns NULL. For notational convenience, let $isort$ be the inverse function of $sorted$, i.e., $isort[j] = c$ if $j = sorted[c]$.

Example 1. The compact prefix tree of the collection $S = \{\text{ACGGAAAT}, \text{GGACTAAACGAC}, \text{ACGACGACA}, \text{ACAAATCTGGTA}, \text{GGACTGTACAA}\}$ is illustrated in Fig. 1.

We first describe an overview of our algorithm for the APSP problem. Our algorithm consists of three steps: preprocessing, matching, and output steps. Our algorithm uses two integers m and B as parameters. (The definitions of m and B are given in Section 2.1. Roughly speaking, m is the length of the shortest input string and B is a logarithmic value of the input size N such that $B \leq m$.)

- In the preprocessing step we construct a compact prefix tree CT for k input strings as in [13]. In addition, we build auxiliary data structures which will be used in the matching step.
- In the matching step we consider each input string S_i separately, and for every suffix s of S_i we need to find all S_j in S such that s is a prefix of S_j (i.e., s is an overlap between S_i and S_j). That is, for every suffix s of S_i we find the interval

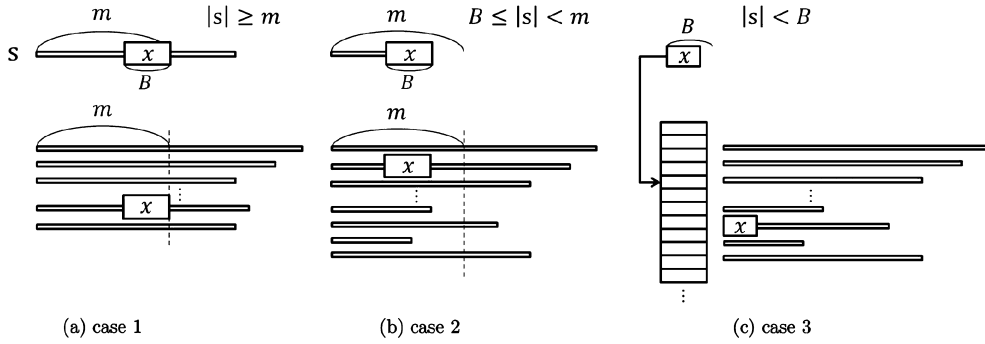


Fig. 2. Three cases of the matching step.

$[a, b]$ in the compact prefix tree CT such that $S_{sorted[a]}, \dots, S_{sorted[b]}$ have s as their prefixes. If the interval $[a, b]$ is not empty, we insert tuple $(a, b, |s|)$ into $oList[i]$. However, if we find the interval $[a, b]$ by calling $Find(s)$ for every suffix s of S_i , it will take too much time. We reduce the number of calls to $Find$ by dividing the suffixes of S_i into three cases and using different techniques for the cases.

- In case 1, we consider each suffix s of S_i such that $|s| \geq m$. If $s[m - B + 1..m]$ appears in $S_j[m - B + 1..m]$ for some j (see Fig. 2 (a)), we call $Find(s)$; otherwise, it is guaranteed that s doesn't appear as a prefix of input strings and so we don't need to call $Find(s)$.
- In case 2, we consider each suffix s such that $B \leq |s| < m$. If $s[|s| - B + 1..|s|]$ appears in $S_j[|s| - B + 1..|s|]$ for some j (see Fig. 2 (b)), we call $Find(s)$; otherwise, s doesn't appear as a prefix of input strings.
- In case 3, we consider suffixes s such that $|s| < B$. For this case we precompute $Find(s')$ for every string s' of length less than B which appears as a prefix of input strings, and we store them in a table $Bprefix$. Hence there are no calls to $Find$ during the matching step.

By the three techniques above, we can drastically reduce the number of calls to $Find$.

- In the output step we find the longest overlap of S_i and S_j for every j , which is the largest l such that tuple (a, b, l) is in $oList[i]$ and interval $[a, b]$ contains $isort[j]$.

2.1. Preprocessing step

In the preprocessing step, we build data structures: a compact prefix tree CT , $sorted$, and auxiliary data structures qrm , $qList$, and $Bprefix$.

We construct a compact prefix tree CT by inserting input strings one by one into the tree while maintaining the lexicographic order of characters for children of each internal node. At the end of the insertions, then, the input strings appear in the leaves of CT in the lexicographic order. Hence $sorted$ and the interval $[a, b]$ of each node can be obtained by traversing the tree.

The values of two parameters m and B are set as follows. Let m_s be the length of the shortest string among k input strings. We define m as follows. (Recall that k is the number of input strings and N is the sum of lengths of all input strings.)

$$m = \begin{cases} m_s & \text{if } \frac{N}{c_1 k} \leq m_s \leq \frac{N}{c_2 k} \\ \frac{N}{c_1 k} & \text{if } m_s < \frac{N}{c_1 k} \\ \frac{N}{c_2 k} & \text{if } m_s > \frac{N}{c_2 k}, \end{cases} \quad (1)$$

for some constants c_1 and c_2 , i.e., $m = \Theta(N/k)$. We define $B = \min(\log_{|\Sigma|} 2mk, m)$, and a string of length B will be called a *block*.

We will use an auxiliary data structure qrm for case 1 in Fig. 2. Let $m' = \max(m, om)$, and let k' be the number of input strings whose length is at least m' . Because we do not need to deal with input strings whose length is less than om , we use m' rather than m . Let $S' = \{S'_1, S'_2, \dots, S'_{k'}\}$ be the collection of the length- m' prefixes of input strings whose length is at least m' . For example, S' is the collection of the strings inside the rectangle in Fig. 3. For every string x of length B , $qrm[f(x)]$ is the rightmost occurrence of x in S' , i.e.,

$$qrm[f(x)] = \begin{cases} \max\{q \mid x = S'_i[q - B + 1..q] \text{ for } 1 \leq i \leq k'\} & \text{if } x \text{ appear in } S' \\ B - 1 & \text{otherwise,} \end{cases} \quad (2)$$

$m = 7, B = 3, om = 2$		$\Sigma = \{A, C, G, T\}$	
		$(rank(A) = 0, rank(C) = 1, rank(G) = 2, rank(T) = 3)$	
S'_1	ACGGAAAT	$qrm[f(AAA)] = 7$	$f(AAA) = 0 * 4^2 + 0 * 4^1 + 0 = 0$
S'_2	GGACTAAACGAC	$qrm[f(ACG)] = 6$	$f(ACG) = 0 * 4^2 + 1 * 4^1 + 2 = 6$
S'_3	ACGACGACA	$qrm[f(TTT)] = 2$	$f(TTT) = 3 * 4^2 + 3 * 4^1 + 3 = 63$
S'_4	ACAAATCTGGTA		
S'_5	GGACTGTACAA		

Fig. 3. An example of qrm .

$m = 7, B = 3, om = 2$			
S''_1	ACGGAAAT	$qList[f(AAT)] = \{6\}$	$f(AAT) = 0 * 4^2 + 0 * 4^1 + 3 = 3$
S''_2	GGACTAAACGAC	$qList[f(GAC)] = \{4, 5\}$	$f(GAC) = 2 * 4^2 + 0 * 4^1 + 1 = 33$
S''_3	ACGACGACA	$qList[f(ACA)] = \{3\}$	$f(ACA) = 0 * 4^2 + 1 * 4^1 + 0 = 4$
S''_4	ACAAATCTGGTA	$qList[f(GTA)] = \{\}$	$f(GTA) = 2 * 4^2 + 3 * 4^1 + 0 = 44$
S''_5	GGACTGTACAA	$qList[f(CAA)] = \{4\}$	$f(CAA) = 1 * 4^2 + 0 * 4^1 + 0 = 16$

Fig. 4. An example of $qList$.

where $f(x)$ is a function mapping a string x to an integer used as an index of the qrm table, i.e.,

$$f(x) = \sum_{i=1}^{|x|} rank(x[i])|\Sigma|^{i-1}, \quad (3)$$

where $rank(c)$ is a function mapping a character c to a lexicographic order of c within the range $[0, \Sigma - 1]$. To compute the values of qrm , we scan all blocks (i.e., all substrings of length B) in S' . The table qrm is initially set to $B - 1$. In each position $q = B, B + 1, \dots, m$, we consider k' blocks $x_i = S'_i[q - B + 1..q]$ for $1 \leq i \leq k'$ and set $qrm[f(x_i)]$ to q .

Example 2. Fig. 3 shows the values of qrm for the collection $\mathcal{S}$ in Example 1. Let m , B , and om be 7, 3, and 2, respectively. Then m' is 7. Let $\Sigma = \{A, C, G, T\}$ and $rank(A) = 0$, $rank(C) = 1$, $rank(G) = 2$, and $rank(T) = 3$. For instance, we will compute the values of qrm for three blocks of “AAA”, “ACG”, and “TTT”. The strings inside the rectangle in Fig. 3 constitute S' . The block “AAA” occurs at (ending) position 7 of S'_1 and position 5 of S'_4 , and thus its rightmost occurrence in S' is 7, as shown in the figure. Similarly, the rightmost occurrence of “ACG” in S' is 6. Since the block “TTT” does not appear in S' , we assign $B - 1 = 2$ to $qrm[f(TTT)]$.

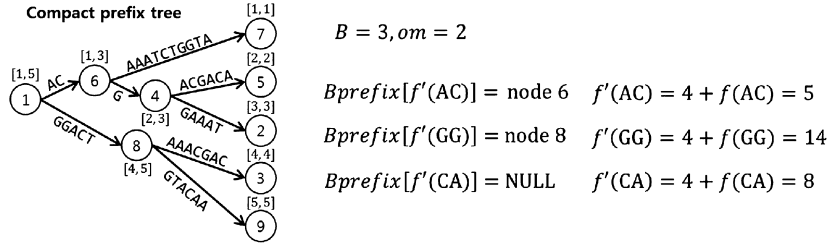
We will use $qList$ for case 2 in Fig. 2. Let $B' = \max(B, om)$ and let k'' be the number of input strings whose length is at least B' . Let $S'' = \{S''_1, S''_2, \dots, S''_{k''}\}$ be the collection of input strings whose length is at least B' . For the last block x of S''_j for every $1 \leq j \leq k''$, $qList[f(x)]$ is defined as a list of all distinct positions q such that $S''_i[q - B + 1..q] = x$ for $1 \leq i \leq k''$ and $B' \leq q < m$. These ranges of i and q are illustrated as a rectangle in Fig. 4.

If $om \geq m$, we do not compute $qList$ because case 2 finds overlaps whose length is less than m . If $om < m$, we compute $qList$ by scanning all blocks in S'' as follows. We define a temporary table T that indicates whether a block x appears at the end of some input string. That is, if $|S_j| \geq B$ and $x = S_j[|S_j| - B + 1..|S_j|]$, $T[f(x)] = 1$; otherwise, $T[f(x)] = 0$. In each position q , we consider k'' blocks $x_i = S''_i[q - B + 1..q]$ for $1 \leq i \leq k''$. If $T[f(x_i)] = 1$ and q is not in $qList[f(x_i)]$ (i.e., q is not at the front of $qList[f(x_i)]$), we insert q into $qList[f(x_i)]$.

Example 3. Fig. 4 shows $qList$ for the collection $\mathcal{S}$ in Example 1. Let m , B , and om be 7, 3, and 2, respectively. Then B' is 3. Since we compute $qList$ only for the last blocks of the input strings in S'' (underlined in Fig. 4), Fig. 4 shows all values of $qList$. The block “GAC” occurs at position 4 of both S''_2 and S''_5 , and at position 5 of S''_3 inside the rectangle, and thus $qList[f(GAC)]$ is $\{4, 5\}$, as shown in the figure. Since the block “GTA” does not appear inside the rectangle, $qList[f(GTA)]$ is empty. Similarly, $qList$ for other blocks can be computed.

We will use $Bprefix$ for case 3 in Fig. 2. Consider every string s such that $om \leq |s| < B$. If s is a prefix of some input string (i.e., s appears as a prefix in CT), $Bprefix[f'(s)]$ is a pointer to the node v nearest to the root of CT such that s is a prefix of the string on the path from the root to v , where $f'(s)$ is a function mapping a string s to an integer:

$$f'(s) = \sum_{i=1}^{|s|-1} |\Sigma|^i + f(s). \quad (4)$$

Fig. 5. An example of *Bprefix*.**Algorithm 1** Fast algorithm for the APSP problem.

```

1: procedure FASTAPSP( $\{S_1, S_2, \dots, S_k\}, om$ )
2:   Precompute  $m, B, m', B', CT, \text{sorted}, qrm, qList$  and  $Bprefix$ 
3:   for  $i \leftarrow 1$  to  $k$  do ▷ Matching step
4:      $p \leftarrow 1$  ▷ Case 1
5:     while  $p \leq |S_i| - m' + 1$  do
6:        $x \leftarrow S_i[p + m - B .. p + m - 1]$ 
7:       if  $qrm[f(x)] = m$  then
8:          $v \leftarrow \text{Find}(S_i[p .. |S_i|])$ 
9:         if  $v \neq \text{NULL}$  then
10:           $oList[i].\text{insert}(v.\text{interval}, |S_i| - p + 1)$ 
11:         $p \leftarrow p + 1$ 
12:       $p \leftarrow p + m - qrm[f(x)]$ 
13:    if  $|S_i| \geq B'$  then ▷ Case 2
14:       $x \leftarrow S_i[|S_i| - B + 1 .. |S_i|]$ 
15:      for each  $q$  in  $qList[f(x)]$  do
16:         $v \leftarrow \text{Find}(S_i[|S_i| - q + 1 .. |S_i|])$ 
17:        if  $v \neq \text{NULL}$  then
18:           $oList[i].\text{insert}(v.\text{interval}, q)$ 
19:    for  $p \leftarrow \max(|S_i| - B + 2, 1)$  to  $|S_i| - om + 1$  do ▷ Case 3
20:       $x \leftarrow S_i[p .. |S_i|]$ 
21:       $v \leftarrow \text{Prefix}[f'(x)]$ 
22:      if  $v \neq \text{NULL}$  then
23:         $oList[i].\text{insert}(v.\text{interval}, |S_i| - p + 1)$ 
24:    Perform output step for  $oList[i]$ 

```

Fig. 6. Pseudocode of the matching step.

Because we cannot distinguish between “A” and “AA” by using f , we employ this function f' for *Bprefix*. Note that $f'(s)$ maps a string s into an integer within the range $[0, \sum_{i=1}^{B-1} |\Sigma|^i - 1]$. If s is not a prefix of the input strings, $Bprefix[f'(x)]$ is set to NULL.

If $om \geq B$, we do not compute *Bprefix* because case 3 finds overlaps whose length is less than B . If $om < B$, we compute *Bprefix* as follows. Initially, all entries of *Bprefix* are set to NULL. We traverse *CT* for all character depths d (i.e., number of characters on the path from the root) such that $om \leq d < B$, and set $Bprefix[f'(s)]$ for string s that appears as a prefix in *CT*.

Example 4. Fig. 5 shows *Bprefix* for the collection $\mathcal{S}$ in Example 1. We compute *Bprefix* for every string s such that $om \leq |s| < B$ (i.e., every string of length 2 in this example). Since “GG” appears as a prefix on the path from the root to node 8, $Bprefix[f'(GG)]$ is node 8. Similarly, $Bprefix[f'(AC)]$ is node 6. For all other strings of length 2, *Bprefix* is NULL.

The space complexity of our algorithm is bounded by the memory space used by k input strings and the data structures *CT*, *sorted*, *qrm*, *qList*, and *Bprefix*. The input uses $N \log |\Sigma|$ bits (i.e., $O(N)$ space), and the data structures use $O(km)$ space, which is $O(N)$ because $km = \Theta(N)$.

2.2. Matching step

In the matching step we consider each input string S_i separately and we need to find $v = \text{Find}(s)$ for every suffix s of S_i . If v is not NULL, we insert tuple $(a, b, |s|)$ into $oList[i]$, where $[a, b]$ is the interval of v . To reduce the number of calls to *Find*, we have the three cases in Fig. 6.

In case 1, we consider suffixes s of S_i such that $|s| \geq m'$ from longest to shortest. Let p be the start position of a current suffix s to be considered. Initially, $p = 1$, and $x = S_i[m - B + 1 .. m]$.

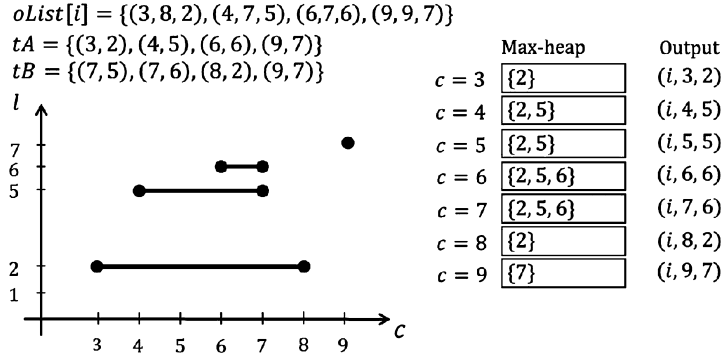


Fig. 7. An example of the output step.

- If $qrm[f(x)]$ is not m , the current suffix s cannot appear in CT as a prefix by definition of qrm , and so we don't need to call $Find(s)$. Moreover, suffixes of S_i starting at positions $p+1, p+2, \dots, p+m-qrm[f(x)]-1$ cannot appear in CT as prefixes. Hence p is updated to $p+m-qrm[f(x)]$.
- If $qrm[f(x)]$ is m , we call $Find(s)$. If $Find$ returns a node v , we insert tuple $(a, b, |s|)$ into $oList[i]$, where $[a, b]$ is the interval of v . If $Find$ returns NULL, we do nothing. Finally, we increase p by 1.

The preprocessing and matching steps of case 1 are essentially the same as those in Wu and Manber's algorithm [18], which is a Boyer–Moore type algorithm [19–21], but cases 2 and 3 are different from Wu and Manber's.

In case 2, we consider suffixes s of S_i such that $B' \leq |s| < m'$. If $om \geq m$ (i.e., $m' = \max(m, om) = om$), we skip case 2. In case 2, therefore, $m' = m$. In this case we consider only the last blocks x of the input strings whose length is at least $B' = \max(B, om)$, i.e., $x = S''_i[|S''_i| - B + 1..|S''_i|]$. Since a position q in $qList[f(x)]$ means that x appears at position q ($B' \leq q < m$) in one of the strings in S'' , the length- q suffix s of S''_i (i.e., $s = S''_i[|S''_i| - q + 1..|S''_i|]$) may appear in CT . Hence, we call $Find(s)$ for every position q in $qList[f(x)]$.

In case 3, we consider suffixes s of S_i such that $|s| < B'$. If $om \geq B$ (i.e., $B' = \max(B, om) = om$), we skip case 3. In case 3, therefore, $B' = B$. For every suffix s of S_i such that $om \leq |s| < B$, we look up $Bprefix[f'(s)]$, which already contains the result of $Find(s)$.

2.3. Output step

In the output step we find the longest suffix of S_i that is a prefix of S_j for every j , whose length is the largest l such that tuple (a, b, l) is in $oList[i]$ and interval $[a, b]$ contains $isort[j]$. (We assume that the output of the APSP problem is a list of three integers (i, j, l) because it is a more compact representation in DNA sequence assembly.) In other words, for every $1 \leq c \leq k$ we want to find the largest l such that tuple (a, b, l) is in $oList[i]$ and interval $[a, b]$ contains c . Then l is the length of the largest overlap of S_i and $S_{sorted[c]}$ and thus we output $(i, sorted[c], l)$.

Suppose that a tuple (a, b, l) in $oList[i]$ represents a closed interval $[a, b]$ parallel to the x -axis with height l , as illustrated in Fig. 7. We scan the intervals from $c = 1$ to k , and maintain the values of l in tuples (a, b, l) such that interval $[a, b]$ contains the current c by using a max-heap. Then for every current c , we output $(i, sorted[c], \text{max value in max-heap})$. This process can be implemented as follows. We first make an array tA of (a, l) 's and an array tB of (b, l) 's from tuples (a, b, l) in $oList[i]$. We sort tA in non-decreasing order of a 's and tB in non-decreasing order of b 's. Finally we increase c from 1 to k , and if c hits a of (a, l) , we insert l into the max-heap, and if c hits b of (b, l) , we delete l from the max-heap. Then $(i, sorted[c], \text{max value in max-heap})$ for every current c is a correct output.

Example 5. Fig. 7 shows an example of the output step. Let $oList[i]$ be a list of $(3, 8, 2)$, $(4, 7, 5)$, $(6, 7, 6)$, and $(9, 9, 7)$. Then tA and tB are computed from $oList[i]$, which are shown in the figure. We also plot a graph which visually shows the elements of $oList[i]$. The status of max-heap and a correct output (i.e., the longest overlap) for each c , $3 \leq c \leq 9$, are shown on the right part of the figure.

Since the number of tuples in $oList[i]$ is at most $|S_i|$, the space usage of $oList[i]$, tA , and tB is $O(|S_i|)$ (thus $O(N)$), and this space for $oList$ can be reused for every $1 \leq i \leq k$.

2.4. Implementation options

The implementation of our algorithm provides several options: *overlap minimum*, *output*, and *parallel* options. The *overlap minimum* option is given by $-om\ i$, where i is the value of overlap minimum om . The *output* option receives 1, 2, or 3 as a parameter. In the case of 1, the program provides the matrix Ov as output. In the case of 2, the program gives the list

Table 1
Real datasets used in experiments.

Parameter	<i>clementina</i>	<i>sinensis</i>	<i>trifoliata</i>	<i>C. elegans</i>	<i>Atta</i>
N	104640576	154995828	46648250	167035020	315387616
k	118365	208909	62344	334465	2835
avg length	884.05	741.93	748.24	499.41	111248
m_s	18	13	89	7	1929
m	55	46	89	31	6952
B	11	12	11	12	12

of three integers $(i, j, Ov[i, j])$ such that $Ov[i, j] \geq om$. In the case of 3, it gives the list of all overlaps for each pair S_i and S_j (not only the longest overlap) like Readjoinder [8] and SOF [13] with $-o 2$. Our program with option 3 presents all the overlaps by outputting $(i, sorted[a], l), (i, sorted[a + 1], l), \dots, (i, sorted[b], l)$ from each tuple (a, b, l) in $oList[i]$ instead of running the output step. Given the number p of threads as a parameter for the *parallel* option, our program is executed in parallel. In the preprocessing step, qrm and $qList$ are computed in parallel. The matching step is also executed in parallel by p threads.

Theorem 1. Algorithm 1 in Fig. 6 finds all overlaps whose length is at least om , and it outputs the longest overlap between S_i and S_j for every pair S_i and S_j .

Proof. In the matching step we consider each input string S_i separately, and for every suffix s of S_i (with $|s| \geq om$) we have to find all S_j in S such that s is a prefix of S_j .

Algorithm 1 divides the suffixes of S_i into three cases. Case 1 considers suffixes s such that $|s| \geq m'$ from longest to shortest. For the longest suffix s (i.e., $p = 1$ in Algorithm 1), we look at the block $x = S_i[m - B + 1..m]$. If $qrm[f(x)] = m$, s may appear in S as prefixes, and so we call $Find(s)$ to find all S_j in S such that s is a prefix of S_j . Otherwise (i.e., $qrm[f(x)] < m$), it is guaranteed that suffixes of S_i starting at positions $p, p + 1, \dots, p + m - qrm[f(x)] - 1$ do not appear in S as prefixes by definition of qrm . Hence p is updated to $p + m - qrm[f(x)]$, and this process is repeated.

Case 2 deals with suffixes s such that $B' \leq |s| < m'$. In this case we consider only the last blocks x of the input strings whose length is at least $B' = \max(B, om)$, i.e., $x = S_i''[|S_i''| - B + 1..|S_i''|]$. Since a position q in $qList[f(x)]$ means that x appears at position q ($B' \leq q < m' = m$) in one of the strings in S'' , the length- q suffix s of S_i'' may appear in CT . Hence, we call $Find(s)$ for every position q in $qList[f(x)]$. It is guaranteed that all other suffixes of S_i do not appear in S as prefixes by definition of $qList$.

Case 3 deals with suffixes s such that $om \leq |s| < B' = B$. We just look up $Bprefix[f'(s)]$, which already contains the result of $Find(s)$. Therefore, we find all overlaps whose length is at least om without missing them.

After the matching step, $oList[i]$ contains all overlaps which are suffixes of S_i . Given $oList[i]$, the output step finds the longest overlap between S_i and $S_{sorted[j]}$ for every j . Therefore, Algorithm 1 finds all overlaps whose length is at least om , and it outputs the longest overlap between S_i and S_j for every pair S_i and S_j . $\square$

3. Experiments

Tustumi et al. [5] and Louza et al. [6] compared only the matching phases of their optimal algorithms for the APSP problem against SOF and Readjoinder, not accounting for the time to build the data structures to store input strings, and SOF and Readjoinder are in general faster than their algorithms. (If the time to build the data structures is included, the gap would be greater.) Among practical algorithms [7–15] for the APSP problem, SOF and Readjoinder show best performances [5, 6, 8, 13]. Therefore, we compared our algorithm¹ with SOF and Readjoinder. Our algorithm and SOF were compiled with `g++` (v. 4.9.2) with the `-O3` optimization flag. Readjoinder (version 1.2) was compiled using the provided Makefile with `"64bit=yes assert=no amalgamation=yes threads=yes"`. For parallel experiments, we used the OpenMP library. All experiments were conducted on a computer with Intel Xeon X5672 CPU, which has 8 cores, 32 GB RAM, and the Linux debian 3.2.0-4-amd64 operating system.

We used two types of datasets which are real and random. The five real datasets are the complete EST databases of *Citrus clementina*,² *Citrus sinensis*,³ *Citrus trifoliata*,² *C. elegans*,³ and *Atta cephalotes*,⁴ which were used as the datasets in the SOF paper [13]. The alphabet of the datasets is $\{A, C, G, T\}$. Table 1 shows specific information of the datasets. Since Readjoinder discards low-quality reads before solving the APSP problem, we removed all comments in the fasta format in order for the three algorithms to consider all reads as input strings for a fair comparison. The random datasets are generated by a program

¹ <http://theory.snu.ac.kr/?p=814>.

² <http://www.citrusgenomedb.org>.

³ <http://www.uni-ulm.de/in/theo/research/seqana>.

⁴ <http://antgenomes.org>.

Table 2

Running time (in second) of algorithms for real datasets.

<i>om</i>	Algorithm	<i>clementina</i>	<i>sinensis</i>	<i>trifoliata</i>	<i>C. elegans</i>	<i>Atta</i>
10	SOF	22.68	44.18	8.07	62.44	23.91
	Readjoiner	114.13	450.48	57.52	231.66	–
	Our1	3.44	6.10	1.50	14.83	0.70
	Our2	5.07	11.21	1.94	14.52	0.86
15	SOF	21.98	40.60	7.70	61.94	24.58
	Readjoiner	50.47	137.54	33.87	84.25	–
	Our1	2.90	4.90	1.37	14.19	0.64
	Our2	3.36	6.44	1.42	13.84	0.73
20	SOF	20.82	36.60	7.35	60.48	23.79
	Readjoiner	11.53	38.42	3.36	44.15	–
	Our1	2.78	4.33	1.30	14.02	0.64
	Our2	2.94	4.75	1.29	13.67	0.69
25	SOF	20.35	35.89	7.33	59.62	23.86
	Readjoiner	7.26	18.8	3.07	27.37	–
	Our1	2.72	4.29	1.29	13.89	0.66
	Our2	2.77	4.42	1.28	13.54	0.66

that gives k strings such that the lengths of the strings follow a normal distribution with mean μ and standard deviation σ and the characters of the strings follow a uniform distribution over the alphabet, where k , μ and σ are parameters given by the user. The alphabet is again $\{A, C, G, T\}$. We generated two datasets *rnd1* and *rnd2*, where *rnd1* has 300000, 1000, and 150 as k , μ , and σ , respectively, and *rnd2* has 1000000, 500, and 100. (In *rnd1*, k is similar to the number of input strings of *C. elegans* and μ is roughly the average length of five real datasets. In *rnd2*, k is larger and μ is smaller than those of *rnd1*.)

We compare the performances of SOF, Readjoiner, and our algorithm for the whole process of the APSP problem, i.e., including the time to build their own data structures, the time for the matching phase, and the time to write the output (For Readjoiner, it is the overlap phase of the whole sequence assembly). The labels of Our1 and Our2 mean our algorithm with option *output* = 2 and *output* = 3, respectively. For SOF, option *-o* 2 is used. Our2, SOF with *-o* 2 and Readjoiner return the same output, which includes all overlaps for each pair of input strings. Our1 solves the APSP problem as it is defined (i.e., it returns the longest overlap for each pair of input strings). Readjoiner doesn't have an option to return the longest overlap for each pair of input strings. SOF finds the longest overlap only when it returns the matrix Ov (with option *-o* 1), but when it returns a list of $(i, j, Ov[i, j])$ as output (with option *-o* 2), it returns all overlaps for each pair of input strings and there is no way to return the longest overlap for each pair. In our algorithm we set c_1 to 16 and c_2 to 8 in all experiments. (We set $c_1 = 2c_2$ and tried $c_2 = 16, 8, 4$, and the values of c_1 and c_2 above showed the best performance.) The values of m and B for real datasets are shown in Table 1.

Table 2 and Fig. 8 show the running time (in second) of each algorithm with the real datasets. We carried out experiments when *om* is 10, 15, 20, and 25. The y -axis (i.e., running time) of Fig. 8 is in log scale. Our algorithm outperforms SOF and Readjoiner in all cases. In the experiment with *Atta*, we obtained a huge speedup compared with SOF because case 1 of the matching step of our algorithm is very effective when m is large. When one of the input strings is very large (its length is over 15 millions in *Atta*), SOF shows a poor performance. In the experiment with *Atta*, Readjoiner stopped and printed “cannot realloc() memory” for all values of *om*. Readjoiner is not efficient for small values of *om* (e.g., *om* = 5). Experimental results show that our algorithm performs well consistently, not depending on one large input string or values of *om*.

Fig. 9 shows the running time of each algorithm with random datasets. Again we did experiments when *om* is 10, 15, 20, and 25. The y -axis (i.e., running time) is in log scale. The performance of our algorithm is better than those of SOF and Readjoiner in all cases. The running time of Readjoiner decreases as *om* increases.

We computed the average speedup of our algorithm Our2 over SOF and Readjoiner for all experiments. The average speedup of Our2 over SOF for all 28 experiments (5 real datasets + 2 random datasets and 4 values of *om*) is about 14 and that of Our2 over Readjoiner for all 24 experiments (4 read datasets + 2 random datasets and 4 values of *om*) is about 18.

Table 3 shows the proportions of the three steps in the running time of our algorithm for real datasets. We used Our1 because the output size of Our2 may be exceedingly large. The proportions for each dataset in Table 3 are average values over the experiments with *om* = 10, 15, 20, and 25. The matching step is the most time-consuming part of our algorithm, accounting for 63.81% on average. The larger the value of m is, the less is the proportion of the matching step, because a large value of m makes the matching step more effective. Table 4 shows the number of calls to *Find* in SOF and our algorithm for real datasets (SOF doesn't use the name *Find*, but it performs the same operation as *Find*). The number of calls to *Find* in our algorithm is on average 2.56% of that of SOF, which is the main reason for huge speedups of our algorithm over SOF and Readjoiner.

Fig. 10 shows the running time of each algorithm with *parallel* options for real datasets. We used different numbers of threads (1, 2, 4, and 8) and a fixed value 15 of *om*, which was the value of *om* in all experiments of the SOF paper [13]. Our algorithm and SOF show reasonable scalability in all experiments. However, SOF does not scale well in the experiment with

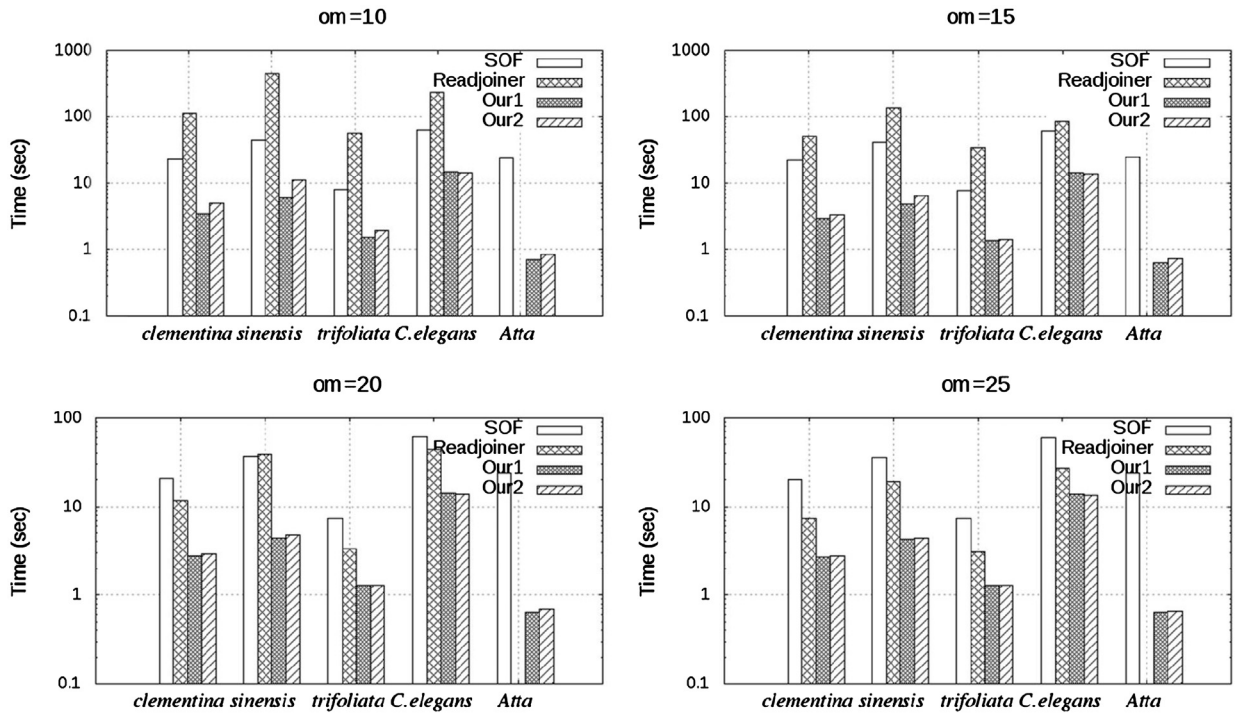


Fig. 8. Running time (in second) of algorithms for real datasets (y-axis in log scale).

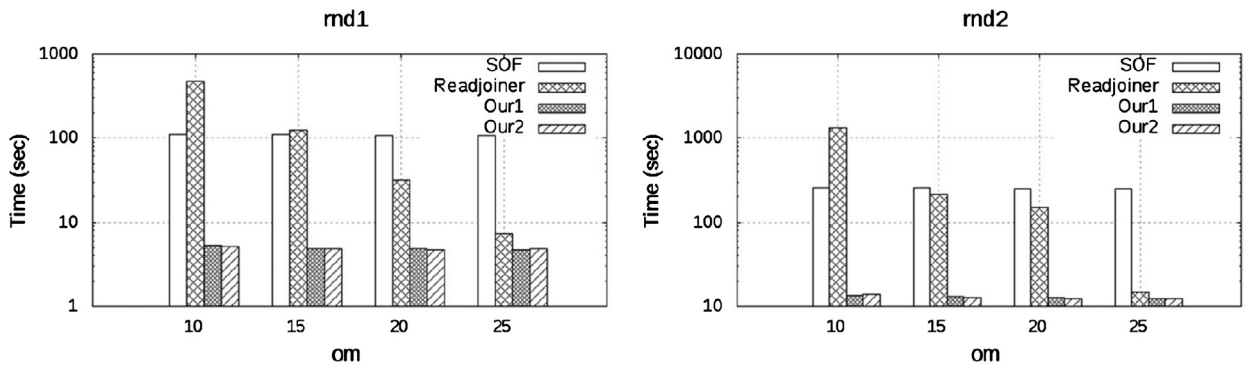


Fig. 9. Running time (in second) of algorithms for random datasets (y-axis in log scale).

Table 3

Proportions of three steps in running time of Our1.

Step	clementina	sinensis	trifoliata	C. elegans	Atta	Total
preproc.	28.22%	18.97%	28.81%	7.80%	56.66%	28.09%
matching	63.93%	69.49%	66.13%	78.98%	40.52%	63.81%
output	7.84%	11.55%	5.06%	13.22%	2.81%	8.10%

Table 4

Number (in thousands) of calls to Find in SOF and our algorithm.

Algorithm	clementina	sinensis	trifoliata	C. elegans	Atta
SOF	102688	152019	45621	161516	315341
Our	1573 (1.53%)	3922 (2.58%)	1577 (3.46%)	7166 (4.44%)	2583 (0.82%)

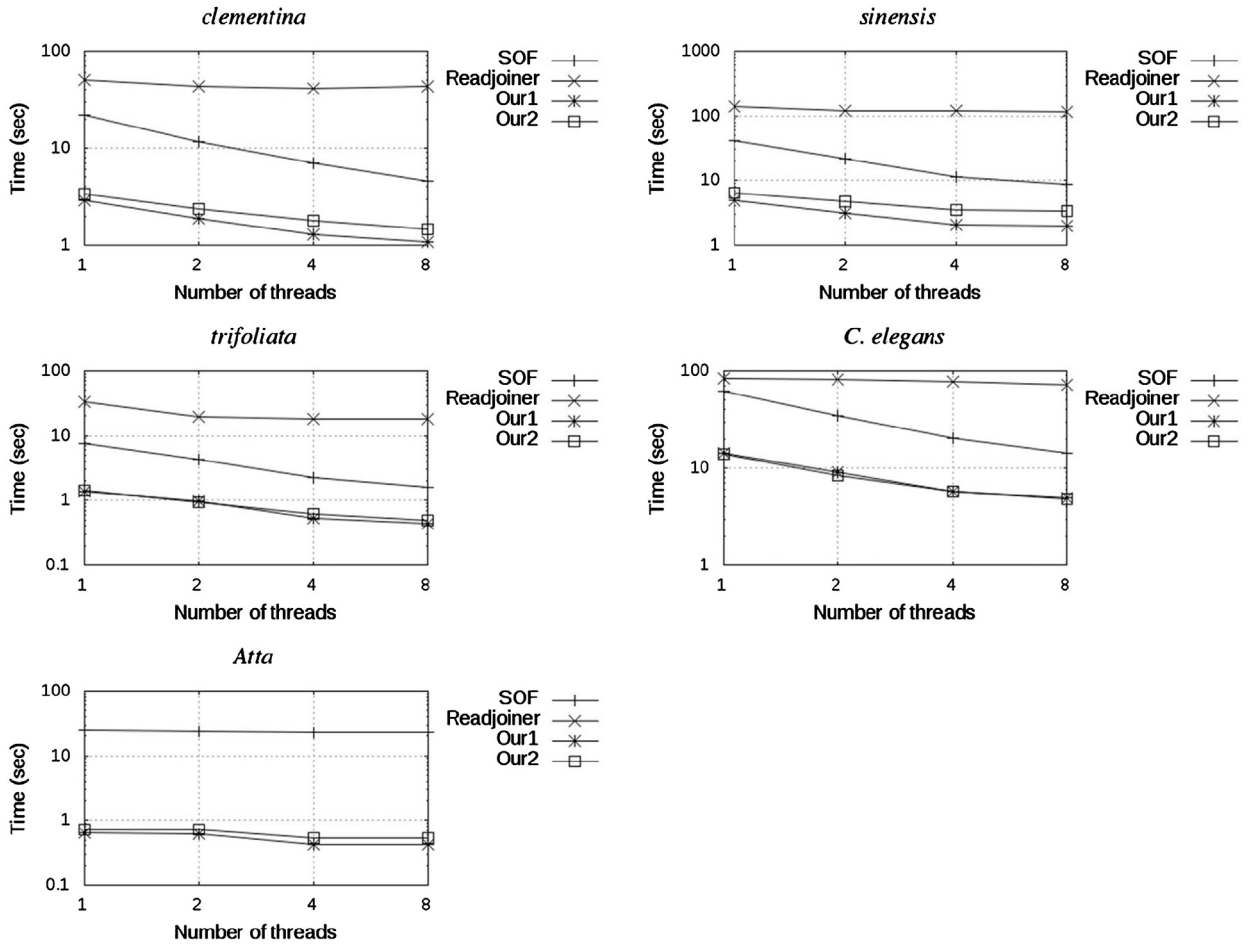


Fig. 10. Running time (in second) of algorithms with *parallel* options 1, 2, 4, and 8 (y-axis in log scale).

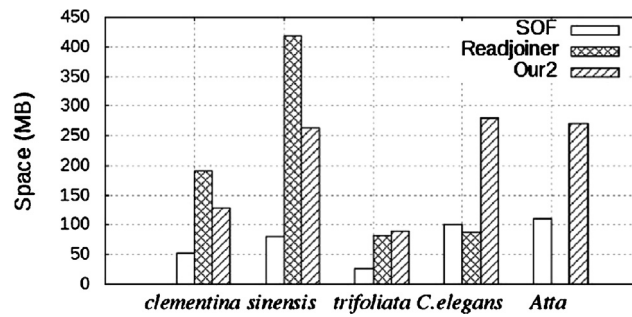


Fig. 11. Peak memory usage of algorithms (y-axis in linear scale).

Atta because SOF has poor scalability when one of the input strings is very large. Readjoinder does not show good scalability in all experiments.

The peak value of memory usage is measured by `/usr/bin/time -v`. Fig. 11 shows the peak memory for SOF, Readjoinder, and Our2 on the real datasets with $om = 15$. Our algorithm uses more memory than SOF in all cases because of additional auxiliary data structures, but the memory usage of our algorithm is still within the optimal bound of $O(N)$ as described in Sections 2.1 and 2.3. Our algorithm uses more memory or less memory than Readjoinder depending on datasets.

Another approach to the APSP problem is to use the Aho–Corasick automaton [17] as mentioned in Section 1. Table 5 shows the construction time of the Aho–Corasick automaton and the total time of SOF with real datasets when om is 15. The construction of the Aho–Corasick automaton (without the matching phase) is slower than the whole SOF in all cases, and so this approach cannot be more efficient than our algorithm.

Table 5

Construction time of Aho–Corasick automaton and total time of SOF (in second).

Algorithm	<i>clementina</i>	<i>sinensis</i>	<i>trifoliata</i>	<i>C. elegans</i>	<i>Atta</i>
SOF	21.98	40.60	7.70	61.94	24.58
AC	27.32	66.18	15.04	62.32	44.65

4. Conclusion

In this paper we have presented a fast algorithm for the all-pairs suffix–prefix problem. The main idea of the algorithm is a combination of a simple but effective data structure for storing input strings and advanced algorithmic techniques for matching to achieve fast running time. Experimental results show that our algorithm runs much faster than previous state-of-the-art algorithms SOF and Readjoiner for the APSP problem. Also we obtain reasonable scalability with a parallel implementation of our algorithm.

Future work is to perform extensive experiments of SOF, Readjoiner, and our algorithm with much larger datasets such as human chromosomes [22] and *Drosophila melanogaster* [23] and analyze characteristics of the three algorithms to obtain possible improvements.

References

- [1] D. Gusfield, G.M. Landau, B. Schieber, An efficient algorithm for the all pairs suffix–prefix problem, *Inform. Process. Lett.* 41 (4) (1992) 181–185.
- [2] E. Ohlebusch, S. Gog, Efficient algorithms for the all-pairs suffix–prefix problem and the all-pairs substring–prefix problem, *Inform. Process. Lett.* 110 (3) (2010) 123–128.
- [3] M.I. Abouelhoda, S. Kurtz, E. Ohlebusch, Replacing suffix trees with enhanced suffix arrays, *J. Discrete Algorithms* 2 (1) (2004) 53–86.
- [4] U. Manber, G. Myers, Suffix arrays: a new method for on-line string searches, *SIAM J. Comput.* 22 (5) (1993) 935–948.
- [5] W.H. Tustumi, S. Gog, G.P. Telles, F.A. Louza, An improved algorithm for the all-pairs suffix–prefix problem, *J. Discrete Algorithms* 37 (2016) 34–43.
- [6] F.A. Louza, S. Gog, L. Zanolto, G. Araujo, G.P. Telles, Parallel computation for the all-pairs suffix–prefix problem, in: *SPIRE 2016*, 2016, pp. 122–132.
- [7] H. Dinh, S. Rajasekaran, A memory-efficient data structure representing exact-match overlap graphs with application for next-generation DNA assembly, *Bioinformatics* 27 (14) (2011) 1901–1907.
- [8] G. Gonnella, S. Kurtz, Readjoiner: a fast and memory efficient string graph-based sequence assembler, *BMC Bioinform.* 13 (1) (2012) 82.
- [9] D. Hernandez, P. François, L. Farinelli, M. Østerås, J. Schrenzel, De novo bacterial genome sequencing: millions of very short reads assembled on a desktop computer, *Genome Res.* 18 (5) (2008) 802–809.
- [10] E.W. Myers, The fragment assembly string graph, *Bioinformatics* 21 (suppl 2) (2005) 79–85.
- [11] M.H. Rachid, Q. Malluhi, M. Abouelhoda, A space-efficient solution to find the maximum overlap using a compressed suffix array, in: *2014 Middle East Conference on Biomedical Engineering (MECBME)*, 2014, pp. 329–333.
- [12] M.H. Rachid, Q. Malluhi, M. Abouelhoda, Using the Sadakane compressed suffix tree to solve the all-pairs suffix prefix problem, *BioMed Res. Int.* 2014 (2014).
- [13] M.H. Rachid, Q. Malluhi, A practical and scalable tool to find overlaps between sequences, *BioMed Res. Int.* 2015 (2015).
- [14] J.T. Simpson, R. Durbin, Efficient construction of an assembly string graph using the FM-index, *Bioinformatics* 26 (12) (2010) 367–373.
- [15] J.T. Simpson, R. Durbin, Efficient de novo assembly of large genomes using compressed data structures, *Genome Res.* 22 (3) (2012) 549–556.
- [16] D. Gusfield, *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, New York, 1997.
- [17] A.V. Aho, M.J. Corasick, Efficient string matching: an aid to bibliographic search, *Commun. ACM* 18 (6) (1975) 333–340.
- [18] S. Wu, U. Manber, A fast algorithm for multi-pattern searching, Technical report TR-94-17, 1–11, 1994.
- [19] R.S. Boyer, J.S. Moore, A fast string searching algorithm, *Commun. ACM* 20 (10) (1977) 762–772.
- [20] M. Crochemore, A. Czumaj, L. Gąsieniec, T. Lecroq, W. Plandowski, W. Rytter, Fast practical multi-pattern matching, *Inform. Process. Lett.* 71 (1999) 107–113.
- [21] R.N. Horspool, Practical fast searching in strings, *Softw. Pract. Exp.* 10 (1980) 501–506.
- [22] Genome Assembly Gold-standard Evaluation (GAGE), <http://gage.cbcb.umd.edu/data/index.html>.
- [23] SRA Run Selector, <https://trace.ncbi.nlm.nih.gov/Traces/study/?acc=SRP029965>.